

Chapter 2. SDK architecture

RP-series microcontrollers are powerful chips, and in particular were designed with a disproportionate amount of system RAM for their point in the microcontroller design space. However it *is* an embedded environment, so RAM, CPU cycles and program space are still at a premium. As a result the trade-offs between performance and other factors (e.g. edge case error handling, runtime vs compile time configuration) are necessarily much more visible to the developer than they might be on other, higher-level platforms.

The intention within the SDK has been for features to just work out of the box, with sensible defaults, but also to give the developer as much control and power as possible (if they want it) to fine tune every aspect of the application they are building and the libraries used.

The next few sections try to highlight some of the design decisions behind the SDK: the how and the why, as much as the what.

NOTE

Some parts of this overview are quite technical or deal with very low-level parts of the SDK and build system. You might prefer to skim this section at first and then read it thoroughly at a later time, after writing a few SDK applications.

2.1. The Build System

The SDK uses **CMake** to manage the build. CMake is widely supported by IDEs (Integrated Development Environments), which can use a **CMakeLists.txt** file to discover source files and generate code autocomplete suggestions. The same **CMakeLists.txt** file provides a terse specification of how your application (or your project with many distinct applications) should be built, which CMake uses to generate a robust build system used by **make**, **ninja** or other build tools. The build system produced is customised for the platform (e.g. Windows, or a Linux distribution) and by any configuration variables the developer chooses.

Section 2.6 shows how CMake can set configuration defines for a particular program, or based on which RP-series microcontroller *board* you are building for, to configure things like default pin mappings and features of SDK libraries. These defines are listed in **Chapter 5**, and Board Configuration files are covered in more detail in **Chapter 8**. Additionally **Chapter 6** describes CMake variables you can use to control the functionality of the build itself.

Apart from being a widely used build system for C/C++ development, CMake is fundamental to the way the SDK is structured, and how applications are configured and built.

Pico Examples: https://github.com/raspberrypi/pico-examples/blob/master/blink_simple/CMakeLists.txt

```

1 add_executable(blink_simple
2     blink_simple.c
3 )
4
5 # pull in common dependencies
6 target_link_libraries(blink_simple pico_stdlib)
7
8 # create map/bin/hex/uf2 file etc.
9 pico_add_extra_outputs(blink_simple)
10
11 # call pico_set_program_url to set path to example on github, so users can find the source
    for an example via picotool
12 example_auto_set_url(blink_simple)
```

Looking again at the `blink_simple` example, we are defining a new executable `blink_simple` with a single source file

`blink_simple.c`, with a single dependency `pico_stdlib`. We also are using a SDK provided function `pico_add_extra_outputs` to ask additional files to be produced beyond the executable itself (`.uf2`, `.hex`, `.bin`, `.map`, `.dis`).

The SDK builds an executable which is "bare metal", i.e. it includes the entirety of the code needed to run on the device (other than certain code contained in the bootrom within the RP-series microcontroller).

`pico_stdlib` is an **INTERFACE** library and provides all the rest of the code and configuration needed to compile and link the `blink` application. You will notice if you watch a build of `blink_simple` (https://github.com/raspberrypi/pico-examples/blob/master/blink_simple/blink_simple.c) that in addition to the single `blink_simple.c` file, the inclusion of `pico_stdlib` causes dozens of other source files to be compiled to flesh out the `blink_simple` application such that it can be run on a RP-series microcontroller.

2.2. Every Library is an INTERFACE Library

All libraries within the SDK are CMake **INTERFACE** libraries. (Note this does not include the C/C++ standard libraries provided by the compiler). Conceptually, a CMake **INTERFACE** library is a collection of:

- Source files
- Include paths
- Compiler definitions (visible to code as `#defines`)
- Compile and link options
- Dependencies (on other **INTERFACE** libraries)

The **INTERFACE** libraries form a tree of dependencies, with each contributing source files, include paths, compiler definitions and compile/link options to the build. These are collected based on the libraries you have listed in your `CMakeLists.txt` file, and the libraries depended on by *those* libraries, and so on recursively. To build the application, each source file is compiled with the combined include paths, compiler definitions and options and linked into an executable according to the provided link options.

When building an executable with the SDK, all of the code for one executable, including the SDK libraries, is (re)compiled for *that* executable from source. Building in this way allows your build configuration to specify customised settings for those libraries (e.g. enabling/disabling assertions, setting the sizes of static buffers), on a *per-application* basis, at compile time. This allows for faster and smaller binaries, in addition of course to the ability to remove support for unwanted features from your executable entirely.

In the example `CMakeLists.txt` we declare a dependency on the (**INTERFACE**) library `pico_stdlib`. This **INTERFACE** library itself depends on other **INTERFACE** libraries (`pico_runtime`, `hardware_gpio`, `hardware_uart` and others). `pico_stdlib` provides all the basic functionality needed to get a simple application running and toggling GPIOs and printing to a UART, and the linker will garbage collect any functions you don't call, so this doesn't bloat your binary. We can take a quick peek into the directory structure of the `hardware_gpio` library, which our `blink_simple` example uses to turn the LED on and off:

```
hardware_gpio
├── CMakeLists.txt
├── gpio.c
├── include
│   └── hardware
│       └── gpio.h
```

Depending on the `hardware_gpio` **INTERFACE** library in your application causes `gpio.c` to be compiled and linked into your executable, and adds the `include` directory shown here to your search path, so that a `#include "hardware/gpio.h"` will pull in the correct header in your code.

INTERFACE libraries also make it easy to aggregate functionality into readily consumable chunks (such as `pico_stdlib`), which don't directly contribute any code, but depend on a handful of lower-level libraries that do. Like a metapackage, this lets you pull in a group of libraries related to a particular goal without listing them all by name.

! IMPORTANT

SDK functionality is grouped into separate **INTERFACE** libraries, and each **INTERFACE** library contributes the code *and* include paths for that library. Therefore, you must declare a dependency on the **INTERFACE** library you need directly (or indirectly through another **INTERFACE** library) for the header files to be found during compilation of your source file (or for code completion in your IDE).

i NOTE

As all libraries within the SDK are **INTERFACE** libraries, we will simply refer to them as libraries or SDK libraries from now on.

2.3. SDK Library Structure

The full API listings are given in [Chapter 4](#); this chapter gives an overview of how SDK libraries are organised, and the relationships between them.

There are a number of layers of libraries within the SDK. This section starts with the highest-level libraries, which can be used in C or C++ applications, and navigates all the way down to the **hardware_regs** library, which is a comprehensive set of hardware definitions suitable for use in Arm assembly as well as C and C++, before concluding with a brief note on how the TinyUSB stack can be used from within the SDK.

2.3.1. Higher-level Libraries

These libraries (**pico_xxx**) provide higher-level APIs, concepts and abstractions that are common to most RP-series microcontroller-based applications. The APIs are listed in [High Level APIs](#). These may be libraries that have cross-cutting concerns between multiple pieces of hardware (for example the **sleep_** functions in **pico_time** need to concern themselves both with the RP-series microcontrollers' timer hardware and with how processors enter and exit low power states), or they may be pure software infrastructure required for your program to run smoothly. This includes libraries for things like:

- Alarms, timers and time functions
- Multi-core support and synchronization primitives
- Utility functions and data structures

These libraries are generally built upon one or more underlying **hardware_** libraries, and often depend on each other.

i NOTE

More libraries are added over time. Certain additional libraries that are not fully supported/stable/documented (e.g. - Audio support (via PIO), DPI/VGA/MIPI Video support (via PIO), file system support, SDIO support via (PIO)) are included in the [Pico Extras](#) GitHub repository.

2.3.2. Runtime Support Libraries

These libraries provide basic application features required for a basic program.

- Runtime startup and initialization functions, e.g. performing minimal hardware initialisation (e.g. default PLL and clock configuration), and calling functions with **constructor** attributes before entering **main()**
- Low level interfacing with the C/C++ runtime library
- Hardware/bootrom accelerated single and double-precision floating point support.

- Compact `printf` support, and `stdio` support via UART, USB, `semihost` and Segger RTT
- On RP2040, language level `/` and `%` support for fast division using RP2040 hardware dividers
- Standard runtime linking setup with default linker scripts

NOTE

There is more high-level discussion of the aggregate library `pico_runtime` in [Section 2.7](#)

2.3.3. Hardware Support Libraries

These are individual libraries (`hardware_XXX`) providing actual APIs for interacting with each piece of physical hardware/peripheral. They are lightweight and provide only thin abstractions. The APIs are listed in [Hardware APIs](#).

These libraries generally provide functions for configuring or interacting with the peripheral at a functional level, rather than accessing registers directly, e.g.:

```
pio_sm_set_wrap(pio, sm, bottom, top);
```

rather than:

```
pio->sm[sm].execctrl =
    (pio->sm[sm].execctrl & ~(PIO_SM0_EXECCTRL_WRAP_TOP_BITS |
    PIO_SM0_EXECCTRL_WRAP_BOTTOM_BITS)) |
    (bottom << PIO_SM0_EXECCTRL_WRAP_BOTTOM_LSB) |
    (top << PIO_SM0_EXECCTRL_WRAP_TOP_LSB);
```

The `hardware_` libraries are intended to have a very minimal runtime cost. They generally do not require any or much RAM, and rarely rely on other runtime infrastructure. In general their only dependencies are the `hardware_structs` and `hardware_regs` libraries that contain definitions of memory-mapped register layout on the RP-series microcontroller. As such they can be used by low-level or other specialized applications that don't want to use the rest of the SDK libraries and runtime.

NOTE

`void pio_sm_set_wrap(PIO pio, uint sm, uint bottom, uint top) {}` is actually implemented as a `static inline` function in https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_pio/include/hardware/pio.h directly as shown above.

Using `static inline` functions is common in SDK header files because such methods are often called with parameters that have fixed known values at compile time. In such cases, the compiler is often able to fold the code down to a single register write (or in this case a read, AND with a constant value, OR with a constant value, and a write) with no function call overhead. This tends to produce much smaller and faster binaries.

2.3.3.1. Hardware Claiming

The hardware layer does provide one small abstraction which is the notion of claiming a piece of hardware. This minimal system allows registration of peripherals or parts of peripherals (e.g. DMA channels) that are in use, and the ability to atomically claim free ones at runtime. The common use of this system - in addition to allowing for safe runtime allocation of resources - provides a better runtime experience for catching software misconfigurations or accidental use of the same piece hardware by multiple independent libraries that would otherwise be very painful to debug.

There are individual claiming/unclaiming methods in the respective `hardware_` libraries.

2.3.4. Hardware Structs Library

The `hardware_structs` library provides a set of C structures which represent the memory mapped layout of the RP-series microcontroller registers in the system address space. This allows you to replace something like the following (which you'd write in C with the defines from the lower-level `hardware_regs`)

```
*(volatile uint32_t *) (PIO0_BASE + PIO_SM1_SHIFTCTRL_OFFSET) |= PIO_SM1_SHIFTCTRL_AUTOPULL_BITS;
```

with something like this (where `pio0` is a pointer to type `pio_hw_t` at address `PIO0_BASE`):

```
pio0->sm[1].shiftctrl |= PIO_SM1_SHIFTCTRL_AUTOPULL_BITS;
```

The structures and associated pointers to memory mapped register blocks hide the complexity and potential error-prone-ness of dealing with individual memory locations, pointer casts and volatile access. As a bonus, the structs tend to produce better code with older compilers, as they encourage the reuse of a base pointer with offset load/stores, instead of producing a 32 bit literal for every register accessed.

The struct headers are named consistently with both the `hardware_` libraries and the `hardware_regs` register headers. For example, if you access the `hardware_pio` library's functionality through `hardware/pio.h`, the `hardware_structs` library (a dependee of `hardware_pio`) contains a header you can include as `hardware/structs/pio.h` if you need to access a register directly, and this itself will pull in `hardware/regs/pio.h` for register field definitions. The PIO header is a bit lengthy to include here. `hardware/structs/pll.h` is a shorter example to give a feel for what these headers actually contain:

SDK: https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2350/hardware_structs/include/hardware/structs/pll.h Lines 27 - 74

```
27 typedef struct {
28     _REG_(PLL_CS_OFFSET) // PLL_CS
29     // Control and Status
30     // 0x80000000 [31] LOCK          (0) PLL is locked
31     // 0x40000000 [30] LOCK_N       (0) PLL is not locked +
32     // 0x00000100 [8]  BYPASS       (0) Passes the reference clock to the output instead of
    the...
33     // 0x0000003f [5:0] REFDIV      (0x01) Divides the PLL input reference clock
34     io_rw_32 cs;
35
36     _REG_(PLL_PWR_OFFSET) // PLL_PWR
37     // Controls the PLL power modes
38     // 0x00000020 [5]  VCPD         (1) PLL VCO powerdown +
39     // 0x00000008 [3]  POSTDIVPD    (1) PLL post divider powerdown +
40     // 0x00000004 [2]  DSMPD        (1) PLL DSM powerdown +
41     // 0x00000001 [0]  PD           (1) PLL powerdown +
42     io_rw_32 pwr;
43
44     _REG_(PLL_FBDIV_INT_OFFSET) // PLL_FBDIV_INT
45     // Feedback divisor
46     // 0x00000fff [11:0] FBDIV_INT (0x000) see ctrl reg description for constraints
47     io_rw_32 fbdiv_int;
48
49     _REG_(PLL_PRIM_OFFSET) // PLL_PRIM
50     // Controls the PLL post dividers for the primary output
51     // 0x00070000 [18:16] POSTDIV1   (0x7) divide by 1-7
52     // 0x00070000 [14:12] POSTDIV2   (0x7) divide by 1-7
53     io_rw_32 prim;
54 }
```

```

55  _REG_(PLL_INTR_OFFSET) // PLL_INTR
56  // Raw Interrupts
57  // 0x00000001 [0]      LOCK_N_STICKY (0)
58  io_rw_32 intr;
59
60  _REG_(PLL_INTE_OFFSET) // PLL_INTE
61  // Interrupt Enable
62  // 0x00000001 [0]      LOCK_N_STICKY (0)
63  io_rw_32 inte;
64
65  _REG_(PLL_INTF_OFFSET) // PLL_INTF
66  // Interrupt Force
67  // 0x00000001 [0]      LOCK_N_STICKY (0)
68  io_rw_32 intf;
69
70  _REG_(PLL_INTS_OFFSET) // PLL_INTS
71  // Interrupt status after masking & forcing
72  // 0x00000001 [0]      LOCK_N_STICKY (0)
73  io_ro_32 ints;
74 } pll_hw_t;

```

The structure contains the layout of the hardware registers in a block, and some defines bind that layout to the base addresses of the *instances* of that peripheral in the RP-series microcontroller global address map.

Additionally, you can use one of the atomic `set`, `clear`, or `xor` address aliases of a piece of hardware to *set*, *clear* or *toggle* respectively the specified bits in a hardware register (as opposed to having the CPU perform a read/modify/write); e.g.:

```
hw_set_alias(pio0->sm[1].shiftctrl = PIO_SM1_SHIFTCTRL_AUTOPULL_BITS;
```

Or, equivalently:

```
hw_set_bits(&pio0->sm[1].shiftctrl, PIO_SM1_SHIFTCTRL_AUTOPULL_BITS);
```

i NOTE

The hardware atomic set/clear/XOR IO aliases are used extensively in the SDK libraries, to avoid certain classes of data race when two cores, or an IRQ and foreground code, are accessing registers concurrently.

i NOTE

On RP-series microcontrollers, the atomic register aliases are a native part of the *peripheral*, not a CPU function, so the system DMA can also perform atomic set/clear/XOR operation on registers.

2.3.5. Hardware Registers Library

The `hardware_regs` library is a complete set of include files for all RP-series microcontroller registers, autogenerated from the hardware itself. This is all you need if you want to peek or poke a memory-mapped register directly, however, higher-level libraries provide more user-friendly ways of achieving what you want in C/C++.

For example, here is a snippet from `hardware/regs/sio.h`:

```
// Description      : Single-cycle IO block
//                    Provides core-local and inter-core hardware for the two
//                    processors, with single-cycle access.
// =====
#ifndef HARDWARE_REGS_SIO_DEFINED
#define HARDWARE_REGS_SIO_DEFINED
// =====
// Register        : SIO_CPUID
// Description      : Processor core identifier
//                    Value is 0 when read from processor core 0, and 1 when read
//                    from processor core 1.
#define SIO_CPUID_OFFSET 0x00000000
#define SIO_CPUID_BITS   0xffffffff
#define SIO_CPUID_RESET   "-"
#define SIO_CPUID_MSB     31
#define SIO_CPUID_LSB     0
#define SIO_CPUID_ACCESS "RO"

#endif
```

These header files are fairly heavily commented (the same information as is present in the datasheet register listings, or the SVD files). They define the offset of every register, and the layout of the fields in those registers, as well as the access type of the field, e.g. "RO" for read-only.

TIP

The headers in `hardware_regs` contain *only* comments and `#define` statements. This means they can be included from assembly files (`.S`, so the C preprocessor can be used), as well as C and C++ files.

2.3.6. TinyUSB Port

In addition to the core SDK libraries, we provide a RP-series microcontroller port of TinyUSB as the standard device and host USB support library within the SDK, and the SDK contains some build infrastructure for easily pulling this into your application.

The `tinysb_dev` or `tinysb_host` libraries within the SDK can be included in your application dependencies in `CMakeLists.txt` to add device or host support to your application respectively. Additionally, the `tinysb_board` library is available to provide the additional "board support" code often used by TinyUSB demos. See the README in [Pico Examples](#) for more information and example code for setting up a fully functional application.

IMPORTANT

RP-series microcontroller USB hardware supports both Host and Device modes, but the two can not be used concurrently. TinyUSB can however also provide support for USB implemented via PIO, which is exposed, if available, via `tinysb_pico_pio_usb`.

2.3.7. FreeRTOS Ports

FreeRTOS ports are available for RP2040 and RP2350 (both under Arm and RISC-V) either on a single core or in dual-core SMP mode.

The SDK does not directly depend on FreeRTOS, but does provide some libraries (particularly for networking) that are designed to be used with FreeRTOS. The [pico-examples repository](#) contains examples that use FreeRTOS, and when building you should set `FREERTOS_KERNEL_PATH`.

The ports are available in the [FreeRTOS-Kernel repository](#).

NOTE

On RP2350, you must initialize the submodules of the FreeRTOS-Kernel repository.

To add FreeRTOS to your project:

1. Copy [FreeRTOS_Kernel_import.cmake](#) into your project.
2. Add a [FreeRTOSConfig.h](#) file to your project.
3. Add FreeRTOS libraries to [CMakeLists.txt](#). The following code snippet shows a basic example:

```
# The following CMake or environment variables should be defined:
# PICO_SDK_PATH = path/to/sdk
# FREERTOS_KERNEL_PATH = path/to/FreeRTOS-Kernel
cmake_minimum_required(VERSION 3.14)

# Pull in SDK (must be before project)
include(pico_sdk_import.cmake)

project(freertos-sample C CXX ASM)

set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)

# Initialize the SDK
pico_sdk_init()

# FREERTOS: include FreeRTOS Kernel libraries
include(FreeRTOS_Kernel_import.cmake)

# assuming you have a sample.c!
add_executable(sample sample.c)

# FREERTOS: FreeRTOSConfig.h needs to be in the include path
target_include_directories(sample PRIVATE ${CMAKE_CURRENT_LIST_DIR})

# FREERTOS: Note, you should pick the FreeRTOS library that suits you best:
#
#           FreeRTOS-Kernel-Heap1 thru FreeRTOS-Kernel-Heap4
#           or
#           FreeRTOS-Kernel-Static
target_link_libraries(sample
    pico_stdlib
    FreeRTOS-Kernel-Heap4
)

pico_add_extra_outputs(sample)
```

2.3.8. Wi-Fi on Pico W

The IP support within the Pico SDK is provided by [lwIP](#). The lwIP raw API is always supported: the full API, including blocking sockets, may be used under FreeRTOS.

There are a number of different library building blocks used within the IP and Wi-Fi support: [pico_lwip](#) for lwIP, [pico_cyw43_driver](#) for the Wi-Fi chip driver, [pico_async_context](#) for accessing the non-thread-safe API (lwIP) in a consistent way whether polling, using multiple cores, or running FreeRTOS.

! IMPORTANT

By default `libcyw43` is licensed for non-commercial use, but users of Raspberry Pi Pico W, Pico WH, or anyone else who builds their product around RP2040 and CYW43439, benefit from a free [commercial-use licence](#).

These libraries can be composed individually by advanced users, but in most common cases they are rolled into a few convenience libraries that you can add to your application's dependencies in `CMakeLists.txt`:

- **`pico_cyw43_arch_lwip_poll`** - For single-core, traditional polling-style access to lwIP on Pico W.
- **`pico_cyw43_arch_threadsafe_background`** - For single or multicore access to lwIP on Pico W, with lwIP callbacks handled in a low-priority interrupt, so no polling is required.
- **`pico_cyw43_arch_lwip_sys_freertos`** - For full access to the lwIP APIs (`NO_SYS=0`) under FreeRTOS.

For fuller details see the [pico_cyw43_arch header file](#). Many examples of using Wi-Fi and lwIP with the Pico SDK may be found in [the pico-examples repository](#).

2.3.9. Bluetooth on Pico W

The Bluetooth support within the Pico SDK is provided by [BTstack](#). Documentation for BTstack can be found [on BlueKitchen's website](#).

! IMPORTANT

In addition to the [standard BTstack licensing](#) terms, a [supplemental licence](#) which covers commercial use of BTstack with Raspberry Pi Pico W or Raspberry Pi Pico WH is provided.

See the [pico-examples](#) repository for Bluetooth examples including the [examples from BTstack](#).

The Bluetooth support within the SDK is composed of multiple libraries:

The `pico_btstack_ble` library adds the support needed for Bluetooth Low Energy (BLE), and the `pico_btstack_classic` library adds the support needed for Bluetooth Classic. You can link to either library individually, or to both libraries enabling the dual-mode support provided by BTstack.

The `pico_btstack_cyw43` library is required for Bluetooth use. It adds support for the Bluetooth hardware on the Pico W, and integrates the BTstack run loop concept with the SDK's `pico_async_context` library allowing for running Bluetooth either via polling or in the background, along with multicore and/or FreeRTOS support.

The following additional libraries are optional:

- **`pico_btstack_sbc_encoder`** - Adds Bluetooth Sub Band Coding (SBC) encoder support.
- **`pico_btstack_sbc_decoder`** - Adds Bluetooth Sub Band Coding (SBC) decoder support.
- **`pico_btstack_bnep_lwip`** - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using lwIP.
- **`pico_btstack_bnep_lwip_sys_freertos`** - Adds Bluetooth Network Encapsulation Protocol (BNEP) support using lwIP with FreeRTOS in `NO_SYS=0` mode.

To use BTstack you must add `pico_btstack_cyw43` and one or both of `pico_btstack_ble` and `pico_btstack_classic` to your application dependencies in your `CMakeLists.txt`. Additionally, you need to provide a `btstack_config.h` file in your source tree and add its location to your include path. For more details, see BlueKitchen's documentation on [how to configure BTstack](#) and the relevant Bluetooth example code in the [pico-examples](#) repository.

The CMake function `pico_btstack_make_gatt_header` can be used to run the BTstack `compile_gatt` tool to make a GATT header file from a BTstack GATT file.

2.4. Directory Structure

We have discussed libraries such as `pico_stdlib` and `hardware_gpio` above. Imagine you wanted to add some code using the RP-series microcontrollers DMA controller to the `hello_world` example in `pico-examples`. To do this you need to add a dependency on another library, `hardware_dma`, which is not included by default by `pico_stdlib` (unlike, say, `hardware_uart`).

You would change your `CMakeLists.txt` to list both `pico_stdlib` and `hardware_dma` as dependencies of the `hello_world` target (executable). (Note the line breaks are not required)

```
target_link_libraries(hello_world
    pico_stdlib
    hardware_dma
)
```

In your source code, you would include the DMA hardware library header as such:

```
#include "hardware/dma.h"
```

Trying to include this header *without* listing `hardware_dma` as a dependency will fail, and this is due to how SDK files are organised into logical functional units on disk, to make it easier to add functionality in the future.

As an aside, this correspondence of `hardware_dma` → `hardware/dma.h` is the convention for *all* toplevel SDK library headers. The library is called `foo_bar` and the associated header is `foo/bar.h`. Some functions may be provided inline in the headers, others may be compiled and linked from additional `.c` files belonging to the library. Both of these require the relevant `hardware_` library to be listed as a dependency, either directly or through some higher-level bundle like `pico_stdlib`.

NOTE

Some libraries have additional headers which are located – for the above example – in `foo/bar/other.h`

You may want to actually find the files in question (although most IDEs will do this for you). The on disk files are actually split into multiple top-level directories. This is described in the next section.

2.4.1. Locations of Files

Whilst you may be focused on building a binary to run specifically on Raspberry Pi Pico, which uses a RP2040, the SDK is structured in a more general way. This is for two reasons:

- 1. To support other future chips in the RP2 family
- 2. To support testing of your code off device (this is *host* mode)

The latter is useful for writing and running unit tests, but also as you develop your software, for example your debugging code or work-in-progress software might be too big or use too much RAM to fit on the device, and much of the software complexity may be non-hardware-specific.

The code is thus split into top-level directories as follows:

Table 1. Top-level directories

Path	Description
<code>src/rp2040/</code>	This contains the <code>hardware_regs</code> and <code>hardware_structs</code> libraries mentioned earlier, along with a handful of other low-level platform libraries, all of which are specific to the RP2040.

Path	Description
<code>src/rp2350/</code>	This contains the <code>hardware_regs</code> and <code>hardware_structs</code> libraries mentioned earlier, along with a handful of other low-level platform libraries, all of which are specific to the RP2350.
<code>src/rp2_common/</code>	This contains the remaining <code>hardware_</code> library implementations for individual hardware components, and <code>pico_</code> libraries or library implementations that are intended specifically for RP-series microcontroller hardware. Libraries are included here even if they are RP2040 or RP2350 specific, if they are considered part of the RP-series microcontroller API proper.
<code>src/common/</code>	This is common code that is not specific to any hardware. This includes utility code, headers providing hardware abstractions for functionality which are simulated in host mode (see below), along with some of the <code>pico_</code> library implementations which, to the extent they use hardware, do so only through the <code>hardware_</code> abstractions.
<code>src/host/</code>	This is a basic set of stub SDK library implementations sufficient to get simple Pico-series device applications running on your computer (Raspberry Pi OS, Linux, macOS or Windows using Cygwin or Windows Subsystem for Linux) for testing purposes. This is not intended to be a fully functional platform, however it is possible to inject additional implementations of libraries to provide more complete functionality.

There is a CMake variable `PICO_PLATFORM` that controls the environment you are building for:

The value of `PICO_PLATFORM` determine which sets of library sources are compiled to build your program. When doing a `PICO_PLATFORM=rp2040` build, you get code from `common`, `rp2_common` and `rp2040`; when doing a host build (`PICO_PLATFORM=host`), you get code from `common` and `host`.

With the advent of RP2350, there are two additional supported `PICO_PLATFORM` values, `rp2350-arm-s` for secure Arm code on RP2350, and `rp2350-riscv` for RISC-V on RP2350. `rp2350` can also be used as a shorthand, but is expanded based on the value of `PICO_DEFAULT_RP2350_PLATFORM`.

TIP

Individual boards support only one of either RP2040 or RP2350. To avoid having to specify `PICO_PLATFORM` in addition to `PICO_BOARD` (see [Section 2.6.1](#)), specifying the latter can now automatically set the former.

Within each top-level directory, the libraries have the following structure (reading `foo_bar` as something like `hardware_uart` or `pico_time`)

```

top-level_dir/
top-level_dir/foo_bar/include/foo_bar.h      # header file
top-level_dir/foo_bar/CMakeLists.txt         # build configuration
top-level_dir/foo_bar/bar.c                  # source file(s)
```

As a concrete example, we can list the `hardware_uart` directory under `pico-sdk/rp2_common` (you may also recall the `hardware_gpio` library we looked at earlier):

```

hardware_uart
├── CMakeLists.txt
├── include
│   └── hardware
│       └── uart.h
└── uart.c
```

`uart.h` contains function declarations and preprocessor defines for the `hardware_uart` library, as well as some inline

functions that are expected to be particularly amenable to constant folding by the compiler. `uart.c` contains the implementations of more complex functions, such as calculating and setting up the divisors for a given UART baud rate.

i NOTE

The directory `top-level_dir/foo_bar/include` is added as an include directory to the *INTERFACE* library `foo_bar`, which is what allows you to include `"foo/bar.h"` in your application

2.5. Conventions for Library Functions

This section covers some common patterns you will see throughout the SDK libraries, such as conventions for function names, how errors are reported, and the approach used to efficiently configure hardware with many register fields without having unreadable numbers of function arguments.

2.5.1. Function Naming Conventions

SDK functions follow a common naming convention for consistency and to avoid name conflicts. Some names are quite long, but that is deliberate to be as specific as possible about functionality, and of course because the SDK API is a C API and does not support function overloading.

2.5.1.1. Name prefix

Functions are prefixed by the library/functional area they belong to; e.g. public functions in the `hardware_dma` library are prefixed with `dma_`. Sometime the prefix refers to a sub group of library functionality (e.g. `channel_config_`)

2.5.1.2. Verb

A verb typically follows the prefix specifying that action performed by the function. `set_` and `get_` (or `is_` for booleans) are probably the most common and should always be present; i.e. a hypothetical method would be `oven_get_temperature()` and `food_add_salt()`, rather than `oven_temperature()` and `food_salt()`.

2.5.1.3. Suffixes

2.5.1.3.1. Blocking/Non-Blocking Functions and Timeouts

Table 2. SDK Suffixes for (non-)blocking functions and timeouts.

Suffix	Param	Description
(none)		The method is non-blocking, i.e. it does not wait on any external condition that could potentially take a long time.
<code>_blocking</code>		The method is blocking, and may potentially block indefinitely until some specific condition is met.
<code>_blocking_until</code>	<code>absolute_time_t until</code>	The method is blocking until some specific condition is met, however it will return early with a timeout condition (see Section 2.5.2) if the <code>until</code> time is reached.
<code>_timeout_ms</code>	<code>uint32_t timeout_ms</code>	The method is blocking until some specific condition is met, however it will return early with a timeout condition (see Section 2.5.2) after the specified number of milliseconds

<code>_timeout_us</code>	<code>uint64_t timeout_us</code>	The method is blocking until some specific condition is met, however it will return early with a timeout condition (see Section 2.5.2) after the specified number of microseconds
--------------------------	----------------------------------	--

2.5.2. Return Codes and Error Handling

As mentioned earlier, there is a decision to be made as to whether/which functions return error codes that can be handled by the caller, and indeed whether the caller is likely to actually do something in response in an embedded environment. Also note that very often return codes are there to handle parameter checking, e.g. when asked to do something with the 27th DMA channel (when there are actually only 12).

In many cases checking for obviously invalid (likely program bug) parameters in (often inline) functions is prohibitively expensive in speed and code size terms, and therefore we need to be able to configure it on/off, which precludes return codes being returned for these exceptional cases.

The SDK follows two strategies:

1. Methods that can legitimately fail at runtime due to runtime conditions e.g. timeouts, dynamically allocated resource, can return a status which is either
 - A `bool` indicating success or not
 - An integer value which, if negative, is standard SDK negative integer return code from the `PICO_ERROR_` family (see `pico_error_code` values in `pico_base`) and if non-negative indicates a successful return. In the latter case the value is either `PICO_OK (0)` or any other positive value if the function actually needs to return something
2. Other issue like invalid parameters, or failure to allocate resources which are deemed program bugs (e.g. two libraries trying to use the same statically assigned piece of hardware) do not affect a return code (usually the functions return `void`) and must cause some sort of exceptional event.

As of right now the exceptional event is a C `assert`, so these checks are always disabled in release builds by default. Additionally most of the calls to `assert` are disabled by default for code/size performance (even in debug builds); You can set `PARAM_ASSERTIONS_ENABLE_ALL=1` or `PARAM_ASSERTIONS_DISABLE_ALL=1` in your build to change the default across the entire SDK, or say `PARAM_ASSERTIONS_ENABLED_I2C=0/1` to explicitly specify the behaviour for the `hardware_i2c` module

In the future we may support calling a custom function to throw an exception in C++ or other environments where stack unwinding is possible.

3. Obviously sometimes the calling code whether it be user code or another higher level function, may not want the called function to assert on bad input, in which case it is the responsibility of the caller to check the validity (there are a good number of API functions provided that help with this) of their arguments, and the caller can then choose to provide a more flexible runtime error experience.
4. Finally, some code may choose to "panic" directly if it detects an invalid state. A "panic" involves writing a message to standard output and then halting (by executing a breakpoint instruction). Panicking is a good response when it is undesirable to even attempt to continue given the current situation.

2.5.3. Use of Inline Functions

SDK libraries often contain a mixture of `static inline` functions in header files, and non-static functions in C source files. In particular, the `hardware_` libraries are likely to contain a higher proportion of inline function definitions in their headers. This is done for speed and code size.

The code space needed to setup parameters for a regular call to a small function in another compilation unit can be substantially larger than the function implementation. Compilers have their own metrics to decide when to inline function implementations at their call sites, but the use of `static inline` definitions gives the compiler more freedom to do this.

This is *particularly* effective in the context of hardware register access because these functions often:

- Have relatively many parameters, which...
- ...are immediately shifted and masked to combine with some register value, and...
- ...are often constants known at compile time

So if the implementation of a hardware access function is inlined, the compiler can propagate the constant parameters through whatever bit manipulation and arithmetic that function may do, collapsing a complex function down to "please write this constant value to this constant address". Again, we are not forcing the compiler to do this, but the SDK consistently tries to give it freedom to do so.

The result is that there is generally no overhead using the lower-level `hardware_` functions as compared with using preprocessor macros with the `hardware_regs` definitions, and they tend to be much less error-prone.

2.5.4. Builder Pattern for Hardware Configuration APIs

The SDK uses a *builder pattern* for the more complex configurations, which provides the following benefits:

1. Readability of code (avoid "death by parameters" where a configuration function takes a dozen integers and booleans)
2. Tiny runtime code (thanks to the compiler)
3. Less brittle (the addition of another item to a hardware configuration will not break existing code)

Take the following hypothetical code example to (quite extensively) configure a DMA channel:

```
int dma_channel = 3;
dma_channel_config config = dma_get_default_channel_config(dma_channel);
channel_config_set_read_increment(&config, true);
channel_config_set_write_increment(&config, true);
channel_config_set_dreq(&config, DREQ_SPI0_RX);
channel_config_set_transfer_data_size(&config, DMA_SIZE_8);
dma_set_config(dma_channel, &config, false);
```

The value of `dma_channel` is known at compile time, so the compiler can replace `dma_channel` with `3` when generating code (*constant folding*). The `dma_` methods are *static inline* methods (from https://github.com/raspberrypi/pico-sdk/blob/master/src/rp2_common/hardware_dma/include/hardware/dma.h) meaning the implementations can be folded into your code by the compiler and, consequently, your constant parameters (like `DREQ_SPI0_RX`) are propagated through this local copy of the function implementation. The resulting code is usually smaller, and certainly faster, than the register shuffling caused by setting up a function call.

The net effect is that the compiler actually reduces all of the above to the following code:

Effective code produced by the C compiler for the DMA configuration

```
*(volatile uint32_t *) (DMA_BASE + DMA_CH3_AL1_CTRL_OFFSET) = 0x00089831;
```

It may seem counterintuitive that building up the configuration by passing a `struct` around, and committing the final result to the IO register, would be so much more compact than a series of direct register modifications using register field accessors. This is because the compiler is customarily forbidden from eliminating IO accesses (illustrated here with a `volatile` keyword), with good reason. Consequently it's easy to unwittingly generate code that repeatedly puts a value into a register and pulls it back out again, changing a few bits at a time, when we only care about the final value of the register. The configuration pattern shown here avoids this common pitfall.

i NOTE

The SDK code is designed to make builder patterns efficient in both *Release* and *Debug* builds. Additionally, even if not all values are known constant at compile time, the compiler can still produce the most efficient code possible based on the values that are known.

2.6. Customisation and Configuration Using Preprocessor variables

The SDK allows use of compile time definitions to customize the behavior/capabilities of libraries, and to specify settings (e.g. physical pins) that are unlikely to be changed at runtime. This allows for much smaller more efficient code, and avoids additional runtime overheads and the inclusion of code for configurations you *might* choose at runtime even though you actually don't (e.g. support PWM audio when you are only using I2S)!

Remember that because of the use of *INTERFACE* libraries, all the libraries your application(s) depend on are built from source for each application in your build, so you can even build multiple variants of the same application with different baked in behaviors.

[Chapter 5](#) has more details and a comprehensive list of the available preprocessor defines, what they do, and what their default values are.

Preprocessor variables may be specified in a number of ways, described in the following sections.

i NOTE

Whether compile time configuration or runtime configuration or both is supported/required is dependent on the particular library itself. The general philosophy however, is to allow sensible default behaviour without the user specifying any settings (beyond those provided by the board configuration).

2.6.1. Preprocessor Variables via Board Configuration File

Many of the common configuration settings are actually related to the particular RP-series microcontroller board being used and include default pin settings for various SDK libraries. The board being used is specified via the `PICO_BOARD` CMake variable which may be specified on the CMake command line or in the environment.

The board configuration provides a header file that specifies defaults if not otherwise specified; for example <https://github.com/raspberrypi/pico-sdk/blob/master/src/boards/include/boards/pico.h> specifies

```
#ifndef PICO_DEFAULT_LED_PIN
#define PICO_DEFAULT_LED_PIN 25
#endif
```

The header `my_board_name.h` is included by all other SDK headers as a result of setting `PICO_BOARD=my_board_name`. You can also create your own board headers.

See [Section 6.2](#) for more full details on `PICO_BOARD` and related CMake variables.

2.6.2. Preprocessor Variables Per Binary or Library via CMake

We could modify the https://github.com/raspberrypi/pico-examples/blob/master/hello_world/CMakeLists.txt with `target_compile_definitions` to specify an alternate set of UART pins to use.

Modified `hello_world CMakeLists.txt` specifying different UART pins

```
add_executable(hello_world
    hello_world.c
)

# SPECIFY two preprocessor definitions for the target hello_world
target_compile_definitions(hello_world PRIVATE
    PICO_DEFAULT_UART_TX_PIN=16
    PICO_DEFAULT_UART_RX_PIN=17
)

# Pull in our pico_stdlib which aggregates commonly used features
target_link_libraries(hello_world pico_stdlib)

# create map/bin/hex/uf2 file etc.
pico_add_extra_outputs(hello_world)
```

The `target_compile_definitions` specifies preprocessor definitions that will be passed to the compiler for every source file in the target `hello_world` (which as mentioned before includes *all* of the sources for all dependent *INTERFACE* libraries). `PRIVATE` is required by CMake to specify the scope for the compile definitions. Note that all preprocessor definitions used by the SDK have a `PICO_` prefix.

2.7. SDK Runtime

For those coming from non-embedded programming, or from other devices, this section will give you an idea of how various C/C++ language level concepts are handled within the SDK

2.7.1. Standard Input/Output (stdio) Support

The SDK provides infrastructure for routing `stdout` and `stdin` to various hardware interfaces, which is provided by the `pico_stdio` library.

- A UART interface specified by a board configuration header. The default for Raspberry Pi Pico is 115200 baud on GPIO0 (TX) and GPIO1 (RX)
- A USB CDC ACM virtual serial port, using TinyUSB's CDC support. The virtual serial device can be accessed through the RP-series microcontrollers' dedicated USB hardware interface, in Device mode
- Minimal semihosting support to direct `stdout` to an external debug host connected via the Serial Wire Debug link on the RP-series microcontroller
- Segger RTT

The support is used via the standard calls like `printf`, `puts`, `getchar`, found in the standard `<stdio.h>` header. By default, `stdout` converts bare linefeed characters to carriage return plus linefeed, for better display in a terminal emulator. This can be disabled at runtime, at build time, or the CR-LF support can be completely removed.

`stdout` is broadcast to all interfaces that are enabled, and `stdin` is collected from all interfaces which are enabled and support input. Since some of the interfaces, particularly USB, have heavy runtime and binary size cost, only the UART interface is included by default. You can add/remove interfaces for a given program at build time with e.g.

```
pico_enable_stdio_usb(target_name, 1) # enable USB CDC stdio for TARGET target_name
```


2.7.2. Printf Support

The SDK runtime packages a lightweight `printf` library by Marco Paland, provided via the `pico_printf` library.

This is a small and largely feature complete implementation, however the C library version (or no `printf` support) can be chosen instead via the CMake function `pico_set_printf_implementation`.

2.7.3. Runtime Initialization and Linking

Using the SDK you can simply write your simple C file with a `main()` method, and a small `CMakeLists.txt` and you can build a binary that works on your RP-series microcontroller.

You can take as much control of this process as you want, but by default, the `pico_runtime` includes these libraries:

- `pico_crt0` - the runtime entry point and default linker scripts which define memory layout
- `pico_standard_link` - configuration for link options and pulling in linker scripts
- `pico_runtime_init` - a default set of initializers to run before reaching `main`.

2.7.4. C-Library Integration

There are a variety of C libraries used by the compilers supported by the SDK. These currently include:

- `newlib`
- `picolibc`
- `llvm-libc`

These each have slightly different integration points for a bare-metal embedded applications, and the SDK runtime takes care of these via the `pico_clib_interface` library.

2.7.5. Floating-point Support

The SDK provides a highly optimized single and double-precision floating point implementation. often significantly faster than the equivalent C library versions. Both basic arithmetic, and scientific functions are provided.

On RP2040 the functions are actually implemented using support provided in the RP2040 bootrom. This means the interface from your code to the ROM floating point library has very minimal impact on your program size, certainly using dramatically less flash storage than including the standard floating point routines shipped with your compiler. The physical ROM storage on the RP-series microcontroller has single-cycle access (with a dedicated arbiter on the RP-series microcontroller busfabric), and accessing code stored here does not put pressure on the flash cache or take up space in memory, so not only are the routines fast, the rest of your code will run faster due them being resident in ROM.

On RP2350 optimized Arm versions of the single-precision floating point functions are provided which use the processors VFP floating point instructions. Optimized versions of the double-precision float point functions are provided using the RP2350's DCP (Double Coprocessor) instructions.

The SDK libraries `pico_float` and `pico_double` provide this support. This includes implementations for all the standard functions from `math.h` as well as additional functions that can be found in `pico/float.h` and `pico/double.h`.

2.7.5.1. Configuration and Alternate Implementations

There are three different floating point implementations provided

Name	Description
<code>default</code>	The default; equivalent to <code>pico</code>
<code>pico</code>	Use the fast/compact SDK/bootrom implementations
<code>compiler</code>	Use the standard compiler provided soft floating point implementations
<code>none</code>	Map all functions to a runtime assertion. You can use this when you know you don't want any floating point support to make sure it isn't accidentally pulled in by some library.

These settings can be set independently for both "float" and "double":

For "float" you can call `pico_set_float_implementation(TARGET NAME)` in your `CMakeLists.txt` to choose a specific implementation for a particular target, or set the CMake variable `PICO_DEFAULT_FLOAT_IMPL` to `pico_float_NAME` to set the default.

For "double" you can call `pico_set_double_implementation(TARGET NAME)` in your `CMakeLists.txt` to choose a specific implementation for a particular target, or set the CMake variable `PICO_DEFAULT_DOUBLE_IMPL` to `pico_double_NAME` to set the default.

TIP

The `pico` floating point library adds very little to your binary size, however it must include implementations for any used functions that are not present in V1 of the bootrom, which is present on early Raspberry Pi Pico boards. If you know that you are only using RP2040s with V2 of the bootrom, then you can specify defines `PICO_FLOAT_SUPPORT_ROM_V1=0` and `PICO_DOUBLE_SUPPORT_ROM_V1=0` so the extra code will not be included. Any use of those functions on a RP2040 with a V1 bootrom will cause a panic at runtime. See the [RP2040 Datasheet](#) for more specific details of the bootrom functions.

2.7.5.1.1. NaN Propagation

The SDK implementation by default treats input NaNs as infinities. If you require propagation of NaN inputs to outputs and NaN outputs for domain errors, then you can set the compile definitions `PICO_FLOAT_PROPAGATE_NANS` and `PICO_DOUBLE_PROPAGATE_NANS` to 1, at the cost of a small runtime overhead.

2.7.6. Hardware Divider

This section applies to RP2040 only.

The SDK includes optimized 32- and 64-bit division functions accelerated by the RP2040 hardware divider, which are seamlessly integrated with the `/` and `%` operators. The SDK also supplies a high-level API which includes combined quotient and remainder functions for 32- and 64-bit, also accelerated by the hardware divider.

See [Figure 1](#) and [Figure 2](#) for 32-bit and 64-bit integer divider comparison.

Figure 1. 32-bit divides by divider size using GCC library (blue), or the SDK library (red) with the RP2040 hardware divider.

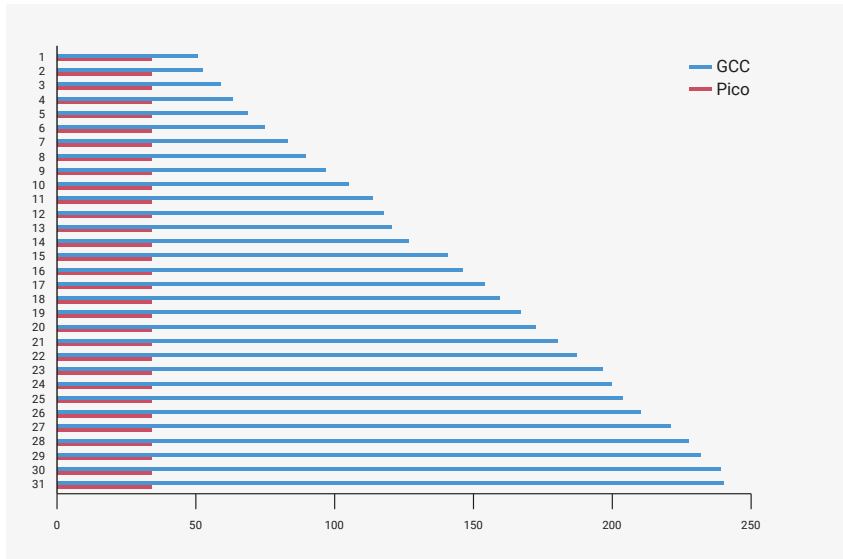
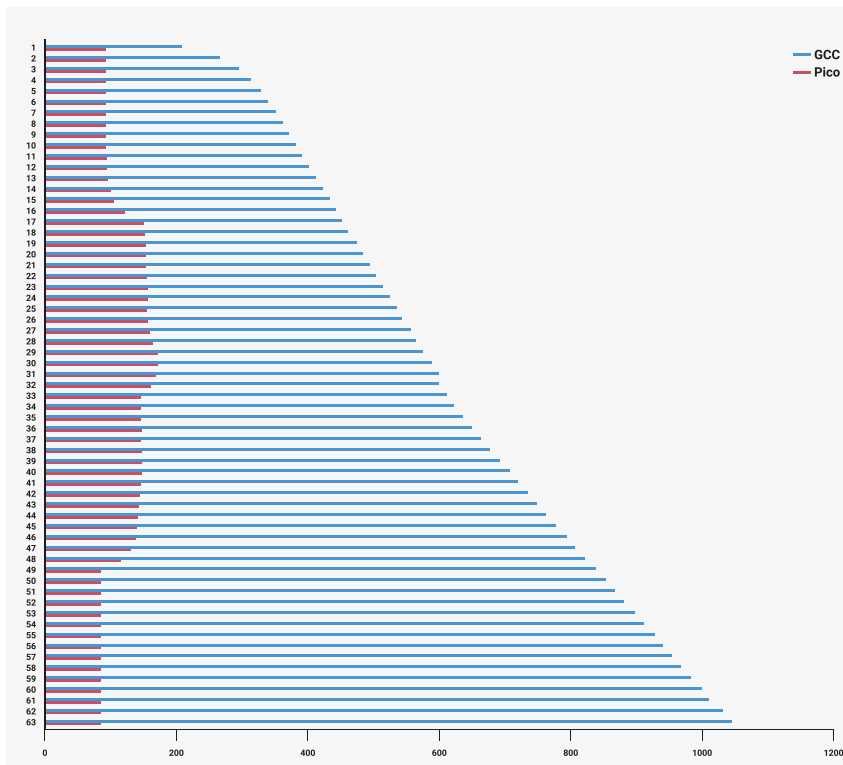


Figure 2. 64-bit divides by divider size using GCC library (blue), or the SDK library (red) with the RP2040 hardware divider.



2.8. Multi-core support

Multi-core support should be familiar to those used to programming with threads in other environments. The second core is just treated as a second *thread* within your application; initially the second core (`core1` as it is usually referred to; the main application thread runs on `core0`) is halted, however you can start it executing some function in parallel from your main application thread.

Core 1 (the second core) is started by calling `multicore_launch_core1(some_function_pointer)`; on core 0, which wakes the core from its low-power sleep state and provides it with its entry point – some function you have provided which hopefully has a descriptive name like `void core1_main() { }`. This function, as well as others such as pushing and popping data through the inter-core mailbox FIFOs, is listed under [pico_multicore](#).

Care should be taken with calling C library functions from both cores simultaneously as they are generally not designed

to be thread safe. You can use the `mutex_` API provided by the SDK in the `pico_sync` library (`mutex`) from within your own code.

NOTE

That the SDK version of `printf` is always safe to call from both cores. `malloc`, `calloc` and `free` are additionally wrapped to make it thread safe when you include the `pico_multicore` as a convenience for C++ programming, where some object allocations may not be obvious.

2.9. Using C++

The SDK has a C style API, however the SDK headers may be safely included from C++ code, and the functions called (they are declared with C linkage).

C++ files are integrated into SDK projects in the same way as C files: listing them in your `CMakeLists.txt` file under either the `add_executable()` entry, or a separate `target_sources()` entry to append them to your target.

To save space, exception handling is disabled by default; this can be overridden with the CMake environment variable `PICO_CXX_ENABLE_EXCEPTIONS=1`. There are a handful of other C++ related `PICO_CXX` vars listed in [Chapter 6](#).

2.10. Supporting both RP2040 and RP2350

The RP2350 supports both Cortex-M33 (Arm) and Hazard3 (RISC-V) processors. As a result the SDK now supports these processors as well as the Cortex-M0 plus processors on the RP2040.

The majority of existing source code using the SDK should compile and run unmodified, even under RISC-V, with the obvious exception of user Arm assembly code, or code interacting with the processor internals.

See [Section 6.2](#) for details of configuring the SDK build for your particular board and RP-series microcontroller platform.

The SDK now supports the compilers listed below, although GCC is still the only *officially* supported compiler as of this SDK 2.0.0.

TIP

If you have the correct compiler in your PATH, then compilation should just work based on your `PICO_PLATFORM` and `PICO_COMPILER` value, however for more control you can set your `PICO_TOOLCHAIN_PATH`. See [Section 6.3](#) for full details, on configuring and finding toolchains

For Arm:

- [GCC arm-none-eabi](#) (`PICO_COMPILER=pico_arm_gcc` - the default for Arm)
 - version 6 onwards for RP2040
 - version 9 onwards for RP2350 since that is the first version that supports the Arm Cortex-M33
- [LLVM Embedded Toolchain For ARM](#) (`PICO_COMPILER=pico_arm_clang`)
 - version 14 onwards
- Pigweed LLVM. This is the vanilla build of LLVM with `llvm-libc` used by [PigWeed](#) (`PICO_COMPILER=pico_arm_clang`)
 - [clang_linux-x86_64](#) (sha256 `e12ee0db9226f5b4a4400c5eb2c0f757d7056181b651622b5453acb00105fd87`)
 - [clang_win-x86_64](#) (sha256 `8c41e8b507f4dfede80842f98a716cac209f552064088fa1b7f4c64a1e547534`)
 - [clang_mac-x86_64](#) (sha256 `1d92f52609d3c1e958fd56f5e9a68ab99b2042ddcc6e90a5eb5009cf7ac4897d`)
 - [clang_mac-aarch64](#) (sha256 `53184680db7e0043a8fba1556c7644b8f5e6c8cdffa4436a92a8e8adb0f45b8d`)

For RISC-V:

- GCC (`PICO_COMPILER=pico_arm_gcc` - the default for RISC-V)

Only very recent versions of GCC fully support the Hazard 3 RISC-V processors, so we recommend the compilers listed below:

- [CORE-V GCC top-of-tree compilers](#)
- Building your own version of GCC 14 as an advanced option. For example, on current Ubuntu:

```
sudo apt-get install autoconf automake autotools-dev curl python3 python3-pip libmpc-
dev libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf libtool
patchutils bc zlib1g-dev libexpat-dev ninja-build git cmake libglib2.0-dev libslirp-dev

sudo mkdir -p /opt/riscv/gcc14-rp2350-no-zcmp

sudo chown -R $(whoami) /opt/riscv/gcc14-rp2350-no-zcmp

git clone https://github.com/riscv/riscv-gnu-toolchain

cd riscv-gnu-toolchain

git clone https://github.com/gcc-mirror/gcc gcc-14 -b releases/gcc-14

./configure --prefix=/opt/riscv/gcc14-rp2350-no-zcmp --with
-arch=rv32ima_zicsr_zifencei_zba_zbb_zbs_zbkb_zca_zcb --with-abi=ilp32 --with-multilib
-generator="rv32ima_zicsr_zifencei_zba_zbb_zbs_zbkb_zca_zcb-ilp32--
;rv32imac_zicsr_zifencei_zba_zbb_zbs_zbkb-ilp32--" --with-gcc-src=`pwd`/gcc-14

make -j$(nproc)
```

2.11. Next Steps

This has been quite a deep dive. If you've somehow made it through this chapter *without* building any software, now would be a perfect time to divert to the [Getting started with Raspberry Pi Pico-series](#) book, which has detailed instructions on connecting to your RP-series microcontroller board and loading an application built with the SDK.

[Chapter 3](#) gives some background on RP-series microcontrollers' unique Programmable I/O subsystem, and walks through building some applications which use PIO to talk to external hardware.

[Chapter 4](#) is a comprehensive listing of the SDK APIs. The APIs are listed according to groups of related functionality (e.g. low-level hardware access).